# Обработка на масиви. Работа с масиви и указатели. Обмен на данни (базови типове, масиви) между функции. Разработване на програми с динамични масиви.

# Съдържание

Динамични масиви ..... 3
Подробно описание на функциите ..... 4
malloc ..... 4
calloc ..... 5
realloc ..... 5
free ..... 6
Пример от реалния живот и реализацията му с код ..... 8
Още примери за работа с динамични масиви ..... 14
Създаване на двумерен масив чрез динамична памет ..... 14
Задачи ..... 18

## Динамични масиви

В езика С няма предвидена опция за работа с масив от динамично променящи се елементи, но използвайки указатели и някои опции на езика може да реализираме такъв похват и да работим с променливи данни.

Динамичният масив представлява масив, чиято дължина не е известна преди началото на изпълнение на програмата и може големината да бъде променяна в хода на изпълнение на програмата. Динамичните масиви се използват в много случаи, когато се налага записването на предварително неизвестен брой елементи, както и когато е нужно намаляване на заеманата памет с цел оптимизация на заеманите системни ресурси. **Може да си направите референция към листовете в Python, Java, C# и така нататък, макар че реализацията не е точно същата.**

За заделянето/презаделянето/освобождаването на динамични масиви се използват функции от стандартната библиотека stdlib.h

Функциите с 4 на брой и те са:

void* malloc(size_t bytes);

void* calloc(size_t elements, size_t bytes_per_element);

void* realloc(void* prev_address, size_t new_size);

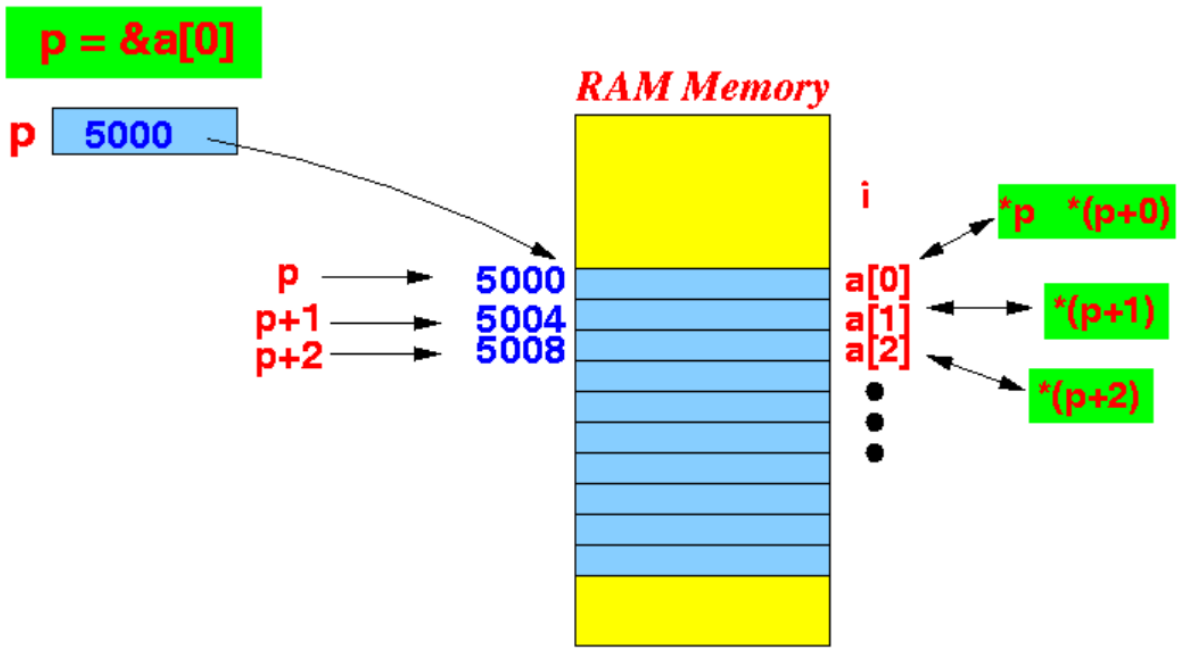
void free(void* array);

В дефиницията на функциите са използвани следните типове:

void* - без типов указател това означава, че няма информация за типа на данните, които са разположени на даден адрес в паметта, **накратко функцията връща пойнтер.**

size_t – предефиниран тип от unsigned long, неотрицателна стойност за задаване на размер.

На следващото изображение може да видим логиката за работа на масив в паметта. В случая, когато заделим блок памет с определен размер чрез функциите по-горе, може този блок да го разбием логически на елементи чрез свойството на указателите да се движат през определен тип данни като по този начин симулираме работата на масив. С други думи работата с динамични масиви е много сходна с тази на работата ни със статични масиви, както и разпределението на елементите в паметта е идентично на това със статичните масиви.




## Подробно описание на функциите

### malloc

Функцията malloc() изисква следните параметри:

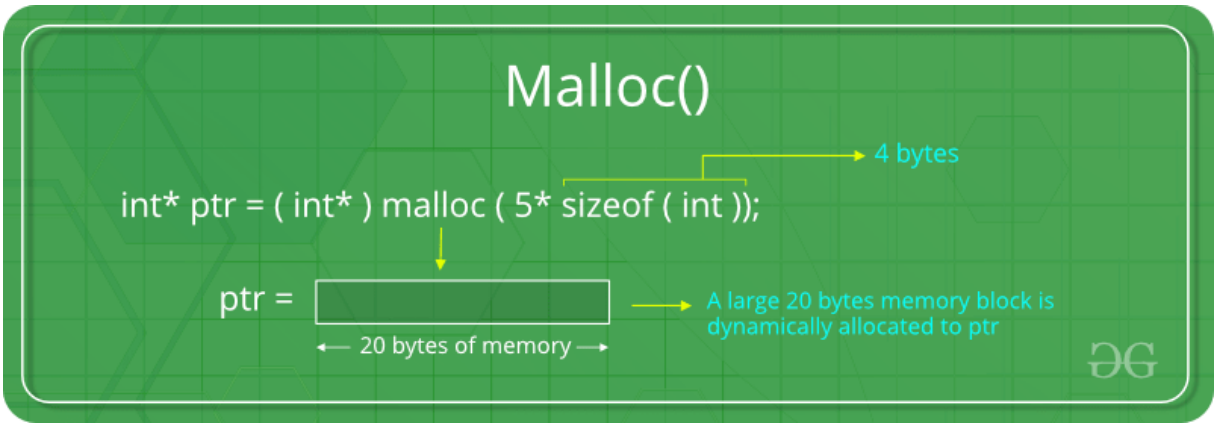
**void* malloc();**

Функцията заделя определения брой байтове и връща указател към адреса на началото на тези заделени данни. „Заделени данни“ означава, че този ресурс памет е специално заделен за използване от програмата и няма да бъде достъпван от външни програми, както и няма променливи, които да използват данните на заделеното пространство. **Malloc НЕ изтрива данните, а само ги заделя. Тоест в създадения ни „масив“ ще имаме случайни стойности.** Програмистът е длъжен да нулира данните предварително за дейности като броене и др.

**Употреба:**

```
int* ptr = (int*)malloc(5 * sizeof(int));
```

**Може би се чудите защо кастваме malloc към int*. Като погледнем по-горе може да**




**видим типът на връщаната стойност за malloc, а именно void*. Тоест за да укажем**

**на компилаторът, че имаме масив от цели числа, трябва да кастнем. Това важи а всички функции свързани с динамично заделяне освен free().**

## calloc

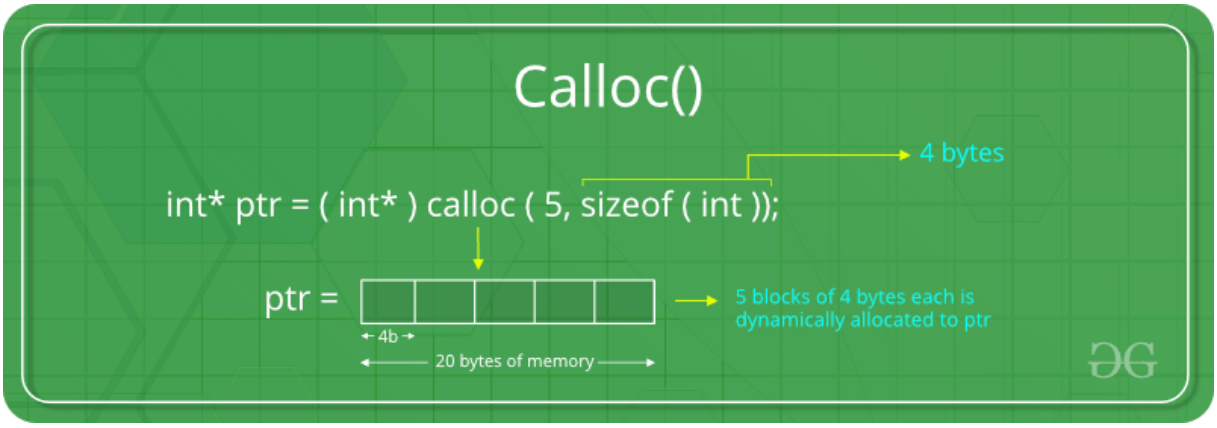
Функцията calloc() изисква следните параметри:

**void* calloc(<брой елементи>, <големина на един елемент>);**

Фукнцията заделя и нулира заделената памет. Тази функция е предназначена за заделяне на памет за масиви, тъй като приема два параметъра – броят на елементите в масива и големината в байтове на всеки един елемент. Резултатът, който връща е указател към първия елемент на масива.

**Употреба:**

```
int* ptr = (int*)calloc(5, sizeof(int));
```




## realloc

Функцията realloc() изисква следните параметри:

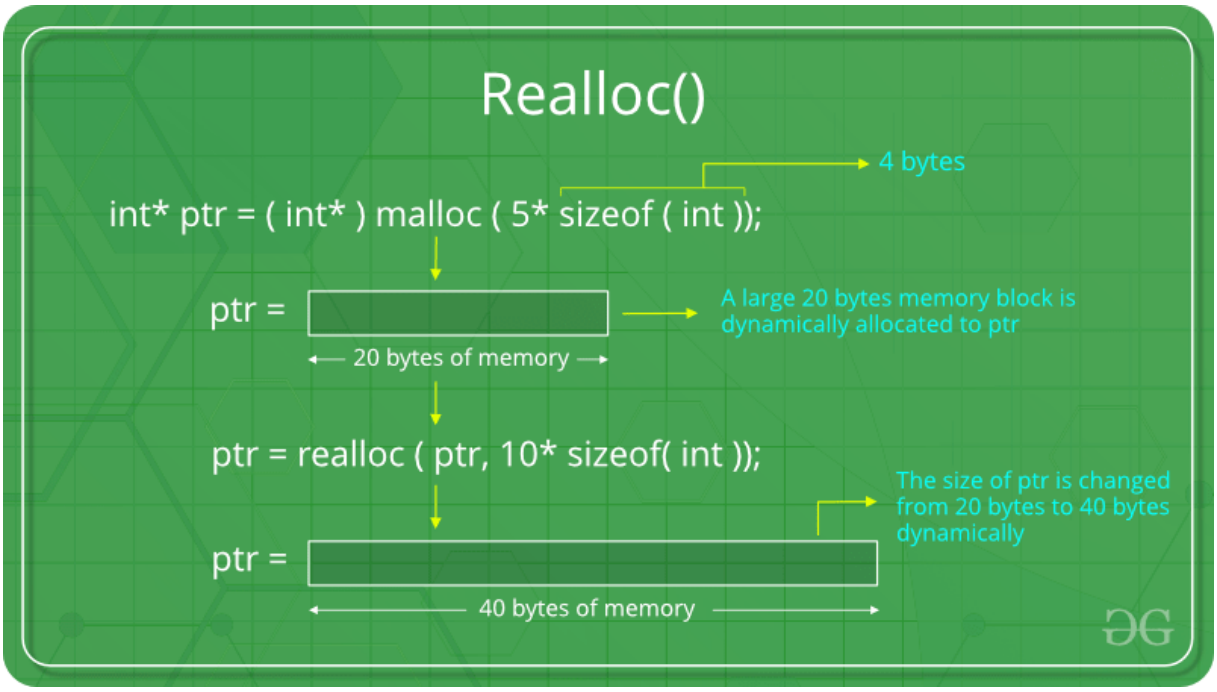
**void* realloc(<старо_начало_на_масива>, <обща големина на масива>);**

Функцията има за цел да промени размера на текущо заделената памет за динамичния масив, като приема два параметъра: старият адрес, където са разположени данните и големината на масива след промяна на размера. Новият размер може да бъде по-голям или по-малък от стария.

Realloc копира (ако е нужно) данните от стария масив в новия масив, до там докъдето новата големина. Функцията връща като резултат новото местоположение на данните. Първоначално функцията проверява дали може да намали/разшири масива на текущото положение в паметта, ако това е невъзможно търси място където може.

**Употреба:**

```
int* ptr = (int*)realloc(ptr, 10*sizeof(int));
```



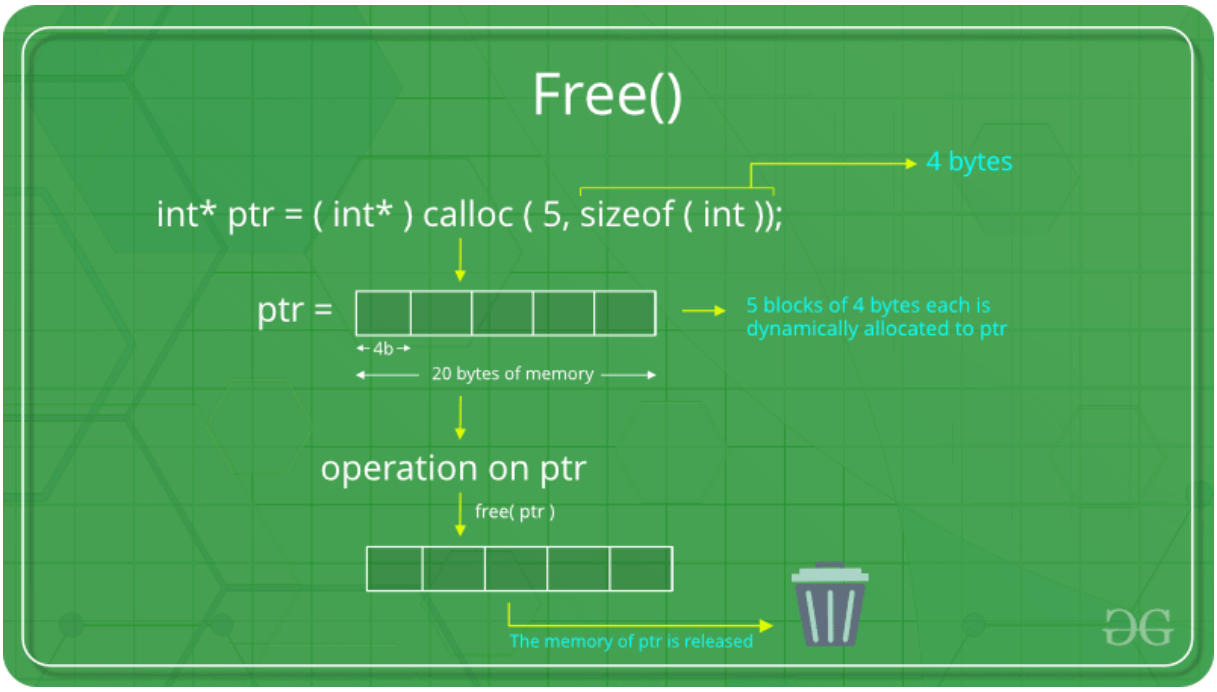

**Всички функции по-горе връщат като резултат NULL, ако не успеят да заделят памет с изисквания размер. Важно е да се проверява дали резултатът им е NULL, понеже може да се появи проблем при записването на данни на нерегламентирани места и да се получи конфликт на данните.**

## free

Тази функция освобождава заетата от по-горните функции памет, като приема само един параметър – адресът на началото на динамичния масива. Добър стил на писане е, ако в края на програмата програмистът освободи заделената памет. **Добре е да се внимава, кога се освобождава памет, понеже двойно освободената памет може да предизвика конфликти, както и не навременното освобождаване да предизвика недостиг на памет.** В други езици от по-висок клас има вградени подсистеми, които се грижат за автоматичното освобождаване на ненужно заета памет. Тъй като програмният един C е предвиден да е език от средно ниво, то тези функции са поверени на програмиста.

### Употреба:

```
int* ptr = (int*)calloc(5, sizeof(int));

free(ptr);
```




## Динамично заделяне и функции

Важно е да споменем, че памет веднъж заделена тя си остава винаги за вашата програма, до момента в който не викнете free. Това ни навежда на мисълта, че ако заделим памет във функция, и не я върнем или освободим вътре във функцията, ние губим достъп до нея. До момента това не беше така, защото всяка една променлива създадена във функцията живее само в нея, но това важи за статични данни, не за памет. Тоест, отново програмистът ще има отговорност за менажиране на тази памет. Пример:

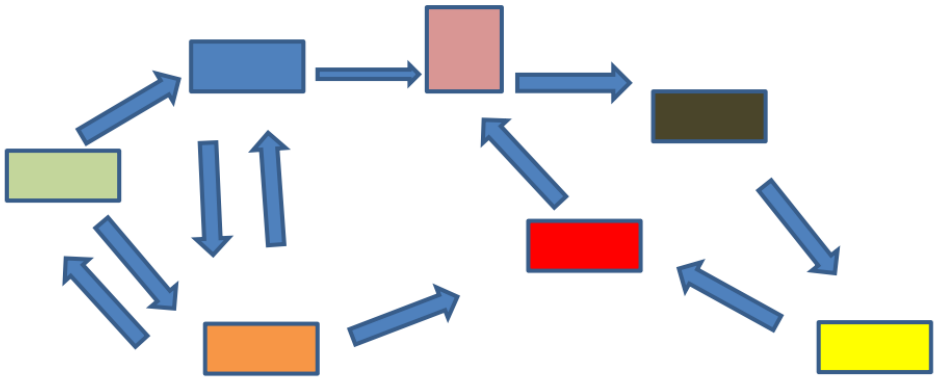
```
int* my_f()
{
   int* ptr = (int*)calloc(5, sizeof(int));
   if(ptr == NULL)
   {
       return NULL;
   }
   ……… // Some code here
   return ptr;
}

int main()
{
   int* bb = my_f();
   // Some code here
   free(bb);
   return 0;
```

```
}
```

## Пример от реалния живот и реализацията му с код

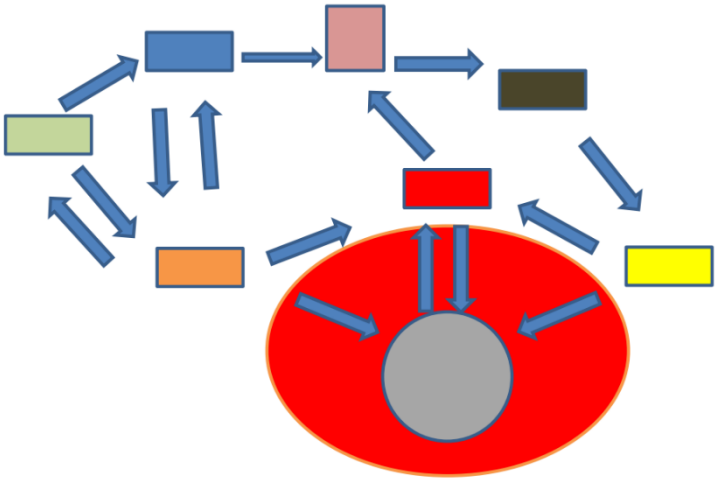
Ако все още не сте разбрали какво е динамичната памет, и къде ни е от полза, то конкретният пример ще внесе яснота.

Представете си, че имаме релсова пътна инфраструктура, която трябва да бъде изградена. От фирма „X“ ни дават нужният маршрут, който ние да симулираме и стигаме до следният вариант:



Както се вижда на графиката по-горе до всяка гара (правоъгълник) може да се стигне по определен път. Гарите са 7 на брой, а изградените релси 11. Правейки си статичен масив и работейки с тези числа, имате фиксирана система, която изпълнява своите функции, но както в реалния живот ако трябва да разширим някой от критериите (релси или гара) няма да можем (до момента масивът ни, който държи информация за гарите и за релсите е с точно определена дължина – 7 за гари и 11 за релси ).

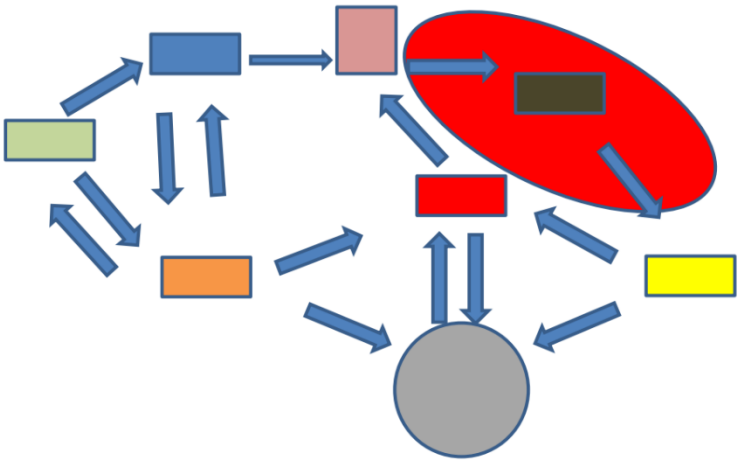
Идва ни заявка за нова гара. Също така за 4 релси които я свързват с другите(в червеното поле).



Ако не съществуваха динамичните данни единственият вариант за промяна и въвеждане на новите данни е пренаписване на цялата програма от програмист. Тук идва и силата на разглежданата техника. Използвайки функция за допълнително заделяне на памет можем

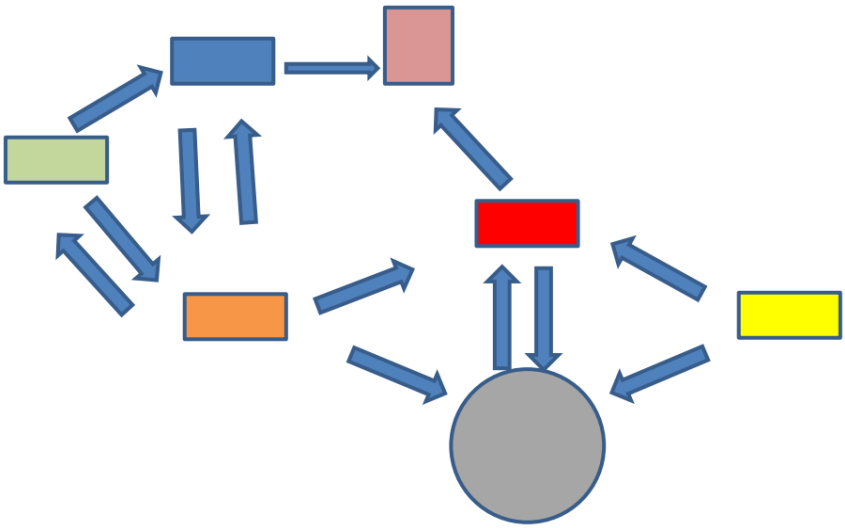
да вземем място за новата гара и четирите релси във вече съществуващите масиви (стига да имаме достатъчно памет). Така само чрез избирането на опция „добави“, администраторът на приложението може да конфигурира цялата си инфраструктура без намесата на трети лица.

Какво ще се случи ако гара или релси са вече неизползваеми? Решаваме да премахнем една гара с пътищата към нея.



В масива от данни, който имаме можем просто да кажем че този елемент го няма вече (да го отбележим с -1), но така остава заделена големината на масива и реално имаме излишък на заето пространство. Тук идва и метода за освобождаване на памет - щом може да бъде заделена динамично, то логично е и да може да бъде освободена по същият начин.

Крайният резултат ще бъде следният:



Примери за решение на задачата с гореописаните функции:

```
#include <stdio.h>      /* printf, scanf, NULL */
#include <stdlib.h>     /* malloc, free, rand */

int main ()
{
  int i,n;
  int * stations;// това е масивът, който ще
//използваме за гарите

  i=7;//началните 7 гари

  stations = (int*) malloc (i*4);//динамично заделяме
//мястото нужно ни за самите гари
  if (stations ==NULL) exit (1);//ако в паметта не е
//заделено нужното място тоест нямаме толкова свободна
//памет

  for (n=0; n<i; n++)
    stations[n]=n;
//както знаем първата гара реално ще е с индекс 0, а
//последната ще е 6

//изкарваме попълнения вече масив
for(n=0;n<7;n++)
printf("%d ",stations[n]);
printf("\n");

int *ptrToStations;
ptrToStations = stations;

       stations = (int*) malloc (++i*4);//увеличаваме
       //броя на 8, като увеличим i
        for(n=0;n<7;n++)
stations[n]=ptrToStations[n];
stations[i-1]=i;

       //имаме вече попълнения масив с новата гара, изкарваме на
       //екрана
for(n=0;n<7;n++)
printf("%d ",stations[n]);
printf("\n");
```

```
  ptrToStations = stations;
 int x=3;//елемента, който ще трием
 for(n=x;n<I;n++)
  ptrToStations[n-1]=ptrToStations[n];
  stations = (int*) malloc (--i*4);//намаляваме
 //броя на 7, като намаляваме i
for(n=0;n<7;n++)
stations[n]=ptrToStations[n];
for(n=0;n<7;n++)
printf("%d ",stations[n]);

  free (stations);
  free (ptrToStations);


  return 0;
}
```

Реализация с calloc():

```
      ptrToStations[n-1]=ptrToStations[n];
      stations = (int*) calloc (--i,sizeof(int));
          for(n=0;n<7;n++)
    stations[n]=ptrToStations[n];
    for(n=0;n<7;n++)
    printf("%d ",stations[n]);
    printf("\n");
 free (stations);
 free (ptrToStations);


 return 0;
}
```

```
#include <stdio.h>      /* printf, scanf, NULL */
#include <stdlib.h>     /* malloc, free, rand */

int main ()
{
  int i,n;
  int * stations
  i=7;
  stations = (int*) calloc (i,sizeof(int));
  if (stationsr==NULL) exit (1);
  for (n=0; n<i; n++)
    stations[n]=n;
for(n=0;n<7;n++)
printf("%d ",stations[n]);
printf("\n");

       int *ptrToStations;
       ptrToStations = stations;
stations = (int*) calloc (++i,sizeof(int));
for(n=0;n<7;n++)
stations[n]=ptrToStations[n];
stations[i-1]=i;

for(n=0;n<7;n++)
printf("%d ",stations[n]);
printf("\n");
        ptrToStations = stations;
       int x=3;
       for(n=x;n<i;n++)
```

Реализация с realloc():

```c
#include <stdio.h>      /* printf, scanf, NULL */
#include <stdlib.h>     /* malloc, free, rand */

int main ()
{
  int count=0,i;
  int *stations=NULL,*ptrToStations=NULL;

for(i=1;i<=7;i++,count++){
ptrToStations=(int*)realloc(stations,count*sizeof(int));

if(ptrToStations!=NULL)//заделили сме памет
     { stations = ptrToStations;
       Stations[count]=i;}
}

for(i=0;i<7;i++)
printf("%d ",stations[i]);
printf("\n");

//добавяне на 8 елемент
ptrToStations=(int*)realloc(stations,++count*sizeof(int));

if(ptrToStations!=NULL)//заделили сме памет
     { stations = ptrToStations;
       Stations[count-1]=count;}
}


for(i=0;i<count;i++)
printf("%d ",stations[i]);

printf("\n");

       int x=3;
       //преместваме елементите с един назад
       for(i=x;i<count;i++)
        ptrToStations[i-1]=ptrToStations[i];

//премахване на 8 елемент
ptrToStations=(int*)realloc(stations,--count*sizeof(int));

if(ptrToStations!=NULL)//заделили сме памет
      stations = ptrToStations;
}


     for(i=0;i<count;i++)
printf("%d ",stations[i]);
printf("\n");
         free (stations);
  free (ptrToStations);


  return 0;
}
```

## Още примери за работа с динамични масиви

Код работещ с динамични масиви и реални числа:

```
int main(int argc, char *argv[])
  {
     int i;

     double* p;

     p = calloc(10, sizeof(double) );  // масив с реални 10 елемента

     for ( i = 0; i < 10; i++ )
    *(p + i) = i;          // стойност i слагаме в масива с индекс i

     for ( i = 0; i < 10; i++ )
    printf("*(p + %d) = %lf\n", i, *(p+i) );


     free(p);      // освбождаване на памет

     putchar('\n');

     p = calloc(4, sizeof(double) );  // правим НОВ масив с 4 елемента

     // ***** Забележете, размера на масива е променен !!! ****

     for ( i = 0; i < 4; i++ )
    *(p + i) = i*i;            // слагаме i*i на индекс i

     for ( i = 0; i < 4; i++ )
    printf("*(p + %d) = %lf\n", i, *(p+i) );

     free(p);      // освобождаваме памет
  }
```

### Създаване на двумерен масив чрез динамична памет

Използване на едномерен масив с аритметика на указатели:

```
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
   int r = 3, c = 4;

   int* ptr = malloc((r * c) * sizeof(int));
```

```
    /* Putting 1 to 12 in the 1D array in a sequence */
```

```
    for (int i = 0; i < r * c; i++)
        ptr[i] = i + 1;

    /* Accessing the array values as if it was a 2D array */
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++)
            printf("%d ", ptr[i * c + j]);
        printf("\n");
    }

    free(ptr);

    return 0;
}
```

Създаване на масив от указатели, които да разширим с новозаделени елементи:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int r = 3, c = 4, i, j, count;

    int* arr[r];
    for (i = 0; i < r; i++)
        arr[i] = (int*)malloc(c * sizeof(int));

    // Note that arr[i][j] is same as *(*(arr+i)+j)
    count = 0;
    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            arr[i][j] = ++count; // Or *(*(arr+i)+j) = ++count

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            printf("%d ", arr[i][j]);

    /* Code for further processing and free the
      dynamically allocated memory */

    for (int i = 0; i < r; i++)
        free(arr[i]);

    return 0;
}
```

Използване на указател към указател като направим динамичен масив с указатели и после заделим памет за всеки от елементите като ползваме двоен указател:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int r = 3, c = 4, i, j, count;

   int** arr = (int**)malloc(r * sizeof(int*));
   for (i = 0; i < r; i++)
      arr[i] = (int*)malloc(c * sizeof(int));

   // Note that arr[i][j] is same as *(*(arr+i)+j)
   count = 0;
   for (i = 0; i < r; i++)
      for (j = 0; j < c; j++)
         arr[i][j] = ++count; // OR *(*(arr+i)+j) = ++count

   for (i = 0; i < r; i++)
      for (j = 0; j < c; j++)
         printf("%d ", arr[i][j]);

   /* Code for further processing and free the
      dynamically allocated memory */

   for (int i = 0; i < r; i++)
      free(arr[i]);

   free(arr);

   return 0;
}
```

Използване на двоен указател и едно викане на malloc():

```
#include<stdio.h>
#include<stdlib.h>

int main()
{
   int r=3, c=4, len=0;
   int *ptr, **arr;
   int count = 0,i,j;

   len = sizeof(int *) * r + sizeof(int) * c * r;
```

```
    arr = (int **)malloc(len);
```

```
    // ptr is now pointing to the first element in of 2D array

    ptr = (int *)(arr + r);

    // for loop to point rows pointer to appropriate location in 2D array
    for(i = 0; i < r; i++)
        arr[i] = (ptr + c * i);

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            arr[i][j] = ++count; // OR *(*(arr+i)+j) = ++count

    for (i = 0; i < r; i++)
        for (j = 0; j < c; j++)
            printf("%d ", arr[i][j]);

    return 0;
}
```

Използване на указател към динамично променлив масив:

Размерността на масива са свързани с типа на променливите. Указателя трябва да се дереференсира чрез (*arr)[i][j]:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int row = 3, col = 4, i, j, count;

    int (*arr)[row][col] = malloc(sizeof *arr);

    count = 0;
    for (i = 0; i < row; i++)
        for (j = 0; j < col; j++)
            (*arr)[i][j] = ++count;

    for (i = 0; i < row; i++)
        for (j = 0; j < col; j++)
            printf("%d ", (*arr)[i][j]);

    free(arr);

    return 0;
}
```

Използване на указател към първия ред на динамичния масив – подобно на горното решение, но може да ползваме синтаксиса за нормален двумерен масив.

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int row = 3, col = 4, i, j, count;

   int (*arr)[col] = calloc(row, sizeof *arr);

   count = 0;
   for (i = 0; i < row; i++)
      for (j = 0; j < col; j++)
         arr[i][j] = ++count;

   for (i = 0; i < row; i++)
      for (j = 0; j < col; j++)
         printf("%d ", arr[i][j]);

   free(arr);

   return 0;
}
```

## Задачи

Решете първите 3 задачи от упражнението за едномерни масиви както и задачата за двумерни от миналото упражнение.